

MINISTRY OF EDUCATION AND RESEARCH
OF THE REPUBLIC OF MOLDOVA

TECHNICAL UNIVERSITY OF MOLDOVA

FACULTY OF COMPUTERS, INFORMATICS AND MICROELECTRONICS
DEPARTMENT OF SOFTWARE AND AUTOMATION ENGINEERING

Report

Laboratory Work No. 11

Race conditions, mutual exclusion locks, counting semaphores, and hardware atomic primitives in Linux

Elaborated by:
st. gr. FAF-241
Tihon Aurelian-Mihai

Verified by:
univ. assist.
Reiţman Patricia

Chişinău – 2026

Contents

INTRODUCTION **3**

1 THEORETICAL ANALYSIS OF RACE CONDITIONS AND SYNCHRONIZATION PRIMITIVES **3**

1.1 Machine instruction decomposition and atomicity violation 4

1.2 Mutual exclusion locks and Linux futex architecture 4

1.3 Counting semaphores and capacity regulation 5

2 EMPIRICAL OBSERVATION OF RACE CONDITIONS AND LOCK PERFORMANCE **5**

2.1 Observation of unprotected concurrent execution 6

2.2 Critical section protection via mutex and binary semaphore 6

2.3 Performance benchmarking across synchronization primitives 7

3 ADVANCED SEMAPHORE SYNCHRONIZATION PATTERNS **7**

3.1 Unidirectional event signaling and execution ordering 8

3.2 Bounded capacity resource regulation: the parking lot problem 8

3.3 Hardware-assisted lockless synchronization via atomic primitives 9

4 CONCLUSION **9**

REFERENCES **11**

ANNEX A: COMPLETE SOURCE CODE LISTINGS **12**

INTRODUCTION

Modern multi-core processor architectures achieve high computational throughput by executing independent execution contexts concurrently across shared physical memory. Within POSIX-compliant multi-threaded environments, threads residing in a single address space share global variables, heap allocations, and open file descriptor tables. While this memory model facilitates high-speed inter-thread communication without the overhead of inter-process memory duplication, it introduces severe hazards: race conditions, non-deterministic state corruption, and lost computational updates. When multiple threads access shared mutable data without coordinated synchronization, the correctness of execution degenerates into an arbitrary function of scheduling latency, cache coherence invalidation, and hardware preemption.

To restore determinism and guarantee memory consistency, operating systems provide fundamental synchronization primitives: mutual exclusion locks (mutexes) and counting semaphores. Understanding the empirical behavior, performance trade-offs, and behavioral nuances of these primitives is an essential requirement for systems programming. A semaphore, initially conceptualized by Edsger Dijkstra, constitutes an integer counter paired with atomic decrement-and-wait and increment-and-wake operations. Depending on its initialization value, a single semaphore primitive serves three distinct synchronization paradigms: a mutual exclusion lock, a unidirectional event signal, and a bounded resource counter.

The objective of this laboratory work is to investigate the mechanics of concurrent thread execution, quantify the hazards of uncoordinated memory access, and evaluate synchronization primitives on GNU/Linux. The investigation proceeds in four systematic phases: first, observing and mathematically analyzing the occurrence of lost updates in an unprotected shared counter (`race.c`); second, restoring deterministic correctness using POSIX mutexes (`mutex.c`) and binary semaphores (`sem.c`), followed by benchmarking runtime latency; third, utilizing a semaphore initialized to zero as an event notification signal to sequence dependent thread actions (`order.c`); and fourth, deploying a counting semaphore to regulate access to finite capacity shared resources (`parking.c`), complemented by an analysis of hardware atomic operations (`atomic.c`). All source code, build scripts, and execution records are maintained in the version control repository hosted on GitHub at `https://github.com/aurelian-th/os-2026-TihonAurelian`.

1 THEORETICAL ANALYSIS OF RACE CONDITIONS AND SYNCHRONIZATION PRIMITIVES

1.1 Machine instruction decomposition and atomicity violation

At the source code level in high-level programming languages such as C, operations on scalar variables frequently appear atomic. For example, incrementing an integer counter via `counter++` is expressed as a single statement. However, contemporary register-based processor architectures (including x86-64 and RISC-V) cannot execute arithmetic transformations directly on system memory. The compilation of `counter++` into native machine code decomposes the abstract operation into three discrete CPU instructions:

$$\text{Instruction 1 (Load):} \quad \text{rax} \leftarrow \text{Memory}[\&counter] \tag{1.1}$$

$$\text{Instruction 2 (Add):} \quad \text{rax} \leftarrow \text{rax} + 1 \tag{1.2}$$

$$\text{Instruction 3 (Store):} \quad \text{Memory}[\&counter] \leftarrow \text{rax} \tag{1.3}$$

Atomicity is defined as the invariant that an operation executes in its entirety without intermediate observation or interruption by concurrent actors. When two threads, Thread A and Thread B, concurrently execute this sequence on a single variable without synchronization, an interleaved schedule can occur:

$$\text{Schedule: } \text{Load}_A \rightarrow \text{Load}_B \rightarrow \text{Add}_A \rightarrow \text{Store}_A \rightarrow \text{Add}_B \rightarrow \text{Store}_B \tag{1.4}$$

If the initial value of `counter` is k, Thread A reads k into its register, and Thread B concurrently reads k into its register. Thread A computes $k+1$ and writes it back to memory. Subsequently, Thread B computes $k+1$ using its stale register value and writes $k+1$ back to memory. Despite two distinct arithmetic additions having executed, the variable advances by only 1, permanently losing one update. Because thread scheduling is non-deterministic, the cumulative error over $2N$ iterations scales with thread collision frequency.

1.2 Mutual exclusion locks and Linux futex architecture

A critical section is defined as any segment of code that accesses shared mutable resources and must not be concurrently executed by more than one thread. Mutual exclusion primitives enforce this isolation by satisfying Dijkstra's four structural criteria: mutual exclusion, absence of deadlocks, progress, and bounded waiting [1].

In POSIX threads (`pthreads`), mutual exclusion is embodied by `pthread_mutex_t`. On modern

Linux kernels, mutexes are implemented using the `futex(2)` (Fast Userspace Mutex) subsystem [2]. The architectural advantage of the futex lies in bifurcating synchronization into an uncontentious userland fast-path and a contentious kernel-level slow-path:

$$\text{Lock State} = \begin{cases} 0, & \text{Unlocked (free)} \\ 1, & \text{Locked, no contention} \\ 2, & \text{Locked, threads waiting in kernel queue} \end{cases} \tag{1.5}$$

When a thread requests a free mutex, it executes an atomic Compare-And-Swap (CAS) instruction entirely in user space. If the lock is unheld, the acquisition completes with zero system call overhead, incurring a latency of only a few processor cycles. Only when contention occurs does the calling thread invoke the `futex(..., FUTEX_WAIT, ...)` system call, causing the Linux kernel scheduler to deschedule the calling thread and place its task struct on a hash bucket wait queue until the lock owner invokes `futex(..., FUTEX_WAKE, ...)`.

1.3 Counting semaphores and capacity regulation

A counting semaphore, denoted by $\mathscr{S}$, consists of a non-negative integer value $\mathscr{V} \in \mathbb{N}_0$, an internal lock protecting the counter, and a wait queue containing suspended thread descriptors [3]. The semaphore supports two invariant atomic primitives:

$$\text{sem_wait}(\mathscr{S}): \quad \text{while } \mathscr{V} == 0 \text{ do sleep}(); \quad \mathscr{V} \leftarrow \mathscr{V} - 1 \tag{1.6}$$

$$\text{sem_post}(\mathscr{S}): \quad \mathscr{V} \leftarrow \mathscr{V} + 1; \quad \text{wakeup}(\text{waiters}) \tag{1.7}$$

The operational behavior of the semaphore is governed entirely by its initialization value $\mathscr{V}_0$, which establishes distinct synchronization semantics as outlined in Table 1.1.

Table 1.1 *Operational Paradigms of Counting Semaphores by Initial Value*

Initial Value ($\mathscr{V}_0$)	**Synchronization Paradigm**	**Operational Function**
$\mathscr{V}_0 = 1$	Mutual Exclusion (Binary Lock)	Protects a critical section; exactly one thread enters
$\mathscr{V}_0 = 0$	Event Signaling / Turnstile	Blocks consumers until a producer signals completion
$\mathscr{V}_0 = K > 1$	Bounded Capacity Counter	Tracks K available resource units; caps concurrent access

2 EMPIRICAL OBSERVATION OF RACE CONDITIONS AND LOCK PERFORMANCE

2.1 Observation of unprotected concurrent execution

To demonstrate the concrete manifestation of race conditions, the program `race.c` was constructed. Two POSIX threads concurrently increment a shared global integer `counter` across $N = 1\,000\,000$ iterations each. The program was compiled with compiler optimizations disabled to ensure that the three-step machine instruction sequence remained uncollapsed in memory. Figure 2.1 illustrates three consecutive terminal executions of `race.c`.

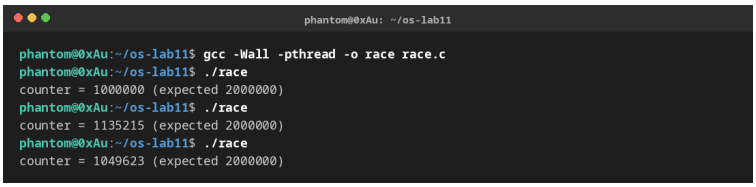


Figure 2.1 – Observation of race conditions and lost additions during unprotected counter execution.

In every execution, the final counter value falls drastically short of the theoretical ground truth (2 000 000), yielding values such as 1 000 000, 1 135 215, and 1 049 623. Furthermore, the variance across executions confirms that thread preemption points and hardware cache line collisions are entirely non-deterministic.

2.2 Critical section protection via mutex and binary semaphore

To eliminate race conditions, the critical section was isolated using two synchronization primitives: a POSIX mutex in `mutex.c` and a binary semaphore initialized to 1 in `sem.c`. In both programs, the three-step increment operation executes within the protected critical section:

```
// In mutex.c:
pthread_mutex_lock(&lock);
counter++;
pthread_mutex_unlock(&lock);

// In sem.c:
sem_wait(&s);
counter++;
sem_post(&s);
```

As demonstrated by the terminal outputs of `mutex` and `sem` displayed in Figure 2.2, both synchro-

nized implementations yielded exactly 2000000 across all executions, verifying that isolating the critical section completely prevents lost additions.



Figure 2.2 – Deterministic verification of counter correctness using mutex and binary semaphore.

2.3 Performance benchmarking across synchronization primitives

While synchronization guarantees correctness, acquiring and releasing locks introduces CPU instruction and memory fence overhead. To evaluate this performance penalty, execution times were benchmarked across the unprotected program (`race`), the mutex implementation (`mutex`), the semaphore implementation (`sem`), and a lock-free hardware atomic implementation (`atomic`), with the captured terminal traces illustrated in Figure 2.3.

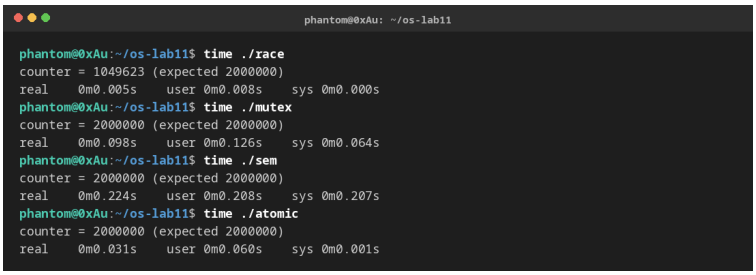


Figure 2.3 – Runtime execution latency and CPU overhead comparison across synchronization mechanisms.

The empirical timing measurements gathered across the four execution models are synthesized in Table 2.1.

Table 2.1*Empirical Latency and Correctness Metrics Across Synchronization Models ($N = 10^6$)*

Mechanism	Real Time (s)	User CPU (s)	System CPU (s)	Correctness Status
Unprotected (`race`)	0.005	0.008	0.000	Incorrect ($< 2 \times 10^6$, lost updates)
POSIX Mutex (`mutex`)	0.098	0.126	0.064	Deterministic (2×10^6)
Binary Semaphore (`sem`)	0.224	0.208	0.207	Deterministic (2×10^6)
Hardware Atomic (`atomic`)	0.031	0.060	0.001	Deterministic (2×10^6, lock-free)

The empirical benchmarks demonstrate that `pthread_mutex` executes over twice as fast as `sem_t` (0.098 s versus 0.224 s). The performance advantage of the mutex stems from the Linux futex fast-path: acquiring an uncontended mutex occurs in user space with minimal register operations, whereas semaphores require managing a generalized integer state and incur additional memory fence synchronization.

3 ADVANCED SEMAPHORE SYNCHRONIZATION PATTERNS

3.1 Unidirectional event signaling and execution ordering

A common synchronization requirement involves enforcing a strict execution order between asynchronous threads without polling. In `order.c`, a consumer thread B must read a string payload only after a producer thread A has populated the buffer. To enforce this sequence, a semaphore `ready` is initialized to 0:

```
sem_init(&ready, 0, 0); // initial value 0: nothing is ready
```

Reader thread B starts first and invokes `sem_wait(&ready)`. Because the counter is 0, thread B immediately suspends. Writer thread A executes `sleep(1)`, copies `"hello"` into the buffer, and invokes `sem_post(&ready)`, incrementing the counter from 0 to 1 and waking thread B, yielding the sequential execution trace depicted in Figure 3.1.

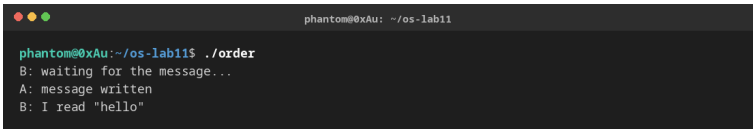


Figure 3.1 – Execution ordering and event signaling trace of `order.c`.

When the initialization value is experimentally changed to 1, thread B passes `sem_wait` immediately without sleeping, reading an uninitialized empty string before thread A completes writing. This verifies that initializing a semaphore to 0 is strictly mandatory when modeling event causality.

3.2 Bounded capacity resource regulation: the parking lot problem

When an application manages a pool of K interchangeable resources, a counting semaphore initialized to K coordinates concurrent access. The program `parking.c` models a parking lot with 3 spaces shared among 6 car threads. The semaphore `places` is initialized to 3:

```
sem_init(&places, 0, 3); // exactly 3 concurrent permits
```

Every car thread invokes `sem_wait(&places)` before entering and `sem_post(&places)` upon leaving. A secondary mutex protects the diagnostic variable `inside` to prevent terminal output interleaving, producing the bounded allocation behavior captured in Figure 3.2.



Figure 3.2 – Multi-car resource allocation trace verifying capacity bounds in `parking.c`.

The trace proves that the number of cars inside the structure fluctuates dynamically between 0 and 3, but never exceeds 3. When `PLACES` is configured to 1, the counting semaphore degenerates into a binary mutex, admitting only one vehicle at a time.

3.3 Hardware-assisted lockless synchronization via atomic primitives

For single scalar variables, modern hardware architectures provide indivisible Read-Modify-Write (RMW) instructions. In `atomic.c`, the global counter is defined as `atomic_long counter = 0;`, and thread increments are executed via:

```
1 atomic_fetch_add(&counter, 1);
```

On x86-64 processors, this translates directly to the instruction `lock incq counter(%rip)`. The `lock` prefix asserts a hardware bus lock or cache coherence lock on the specific cache line, executing the Load-Add-Store sequence as a single indivisible cycle. As recorded in Table 2.1, atomic synchronization achieves deterministic accuracy in 0.031 s, outperforming mutexes by a factor of 3 and semaphores by a factor of 7 without thread context switching or lock contention.

4 CONCLUSION

The experimental investigation conducted in this laboratory work demonstrates the physical reality of race conditions and validates the core synchronization primitives provided by the POSIX standard and the Linux operating system kernel. By dissecting high-level variable increments into their constituent machine instructions, the empirical vulnerability of unsynchronized shared memory was demonstrated. Uncoordinated multi-threaded execution predictably produced lost updates and non-deterministic state corruption across consecutive executions, proving that concurrent memory mutations demand strict synchronization protocols.

The integration of mutual exclusion locks and binary semaphores successfully restored mathematical determinism, ensuring that all increments completed without data loss. Furthermore, comparative benchmarking exposed the substantial latency differences governing synchronization mechanisms. The Linux futex-backed mutex achieved superior throughput relative to POSIX counting semaphores by avoiding kernel transitions on uncontended execution paths, while hardware-level atomic operations demonstrated optimal efficiency by eliminating lock acquisition overhead entirely.

Finally, the deployment of counting semaphores across diverse operational configurations highlighted the versatility of the semaphore abstraction. Initializing the counter to zero established deterministic event sequencing between asynchronous reader and writer threads, while initializing the counter to a positive integer strictly enforced bounded capacity constraints across competing tasks. These foundational primitives establish the theoretical and practical framework required for resolving complex multi-tool concurrency challenges, including producer-consumer buffers, condition variables, and deadlock resolution.

REFERENCES

[1] E. W. Dijkstra, *Cooperating Sequential Processes*. Technological University Eindhoven, The Netherlands, Tech. Rep. EWD-123, 1965.

[2] U. Drepper, "Futexes are strange," Red Hat Inc., Tech. Rep., 2011.

[3] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 10th ed. Hoboken, NJ, USA: John Wiley & Sons, 2018.

[4] IEEE and The Open Group, *The Open Group Base Specifications Issue 7, 2018 Edition (POSIX.1-2017)*, IEEE Std 1003.1-2017, 2018.

ANNEX A: COMPLETE SOURCE CODE LISTINGS

A.1 Source code of the unprotected race condition test utility (`race.c`)

```c
#include <stdio.h>
#include <pthread.h>
#define N 1000000

long counter = 0;

void *add(void *arg) {
    (void)arg;
    for (int i = 0; i < N; i++)
        counter++;
    return NULL;
}

int main(void) {
    pthread_t a, b;
    pthread_create(&a, NULL, add, NULL);
    pthread_create(&b, NULL, add, NULL);
    pthread_join(a, NULL);
    pthread_join(b, NULL);
    printf("counter = %ld (expected %d)\n", counter, 2 * N);
    return 0;
}
```

A.2 Source code of the mutex synchronization utility (`mutex.c`)

```c
#include <stdio.h>
#include <pthread.h>
#define N 1000000

long counter = 0;
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;

void *add(void *arg) {
    (void)arg;
    for (int i = 0; i < N; i++) {
        pthread_mutex_lock(&lock);
        counter++;
        pthread_mutex_unlock(&lock);
    }
    return NULL;
}

int main(void) {
    pthread_t a, b;
    pthread_create(&a, NULL, add, NULL);
    pthread_create(&b, NULL, add, NULL);
    pthread_join(a, NULL);
    pthread_join(b, NULL);
    printf("counter = %ld (expected %d)\n", counter, 2 * N);
    return 0;
}
```

A.3 Source code of the semaphore lock utility (`sem.c`)

```c
#include <stdio.h>
#include <pthread.h>
```

```c
#include <semaphore.h>
#define N 1000000

long counter = 0;
sem_t s;

void *add(void *arg) {
    (void)arg;
    for (int i = 0; i < N; i++) {
        sem_wait(&s);
        counter++;
        sem_post(&s);
    }
    return NULL;
}

int main(void) {
    sem_init(&s, 0, 1);
    pthread_t a, b;
    pthread_create(&a, NULL, add, NULL);
    pthread_create(&b, NULL, add, NULL);
    pthread_join(a, NULL);
    pthread_join(b, NULL);
    printf("counter = %ld (expected %d)\n", counter, 2 * N);
    sem_destroy(&s);
    return 0;
}
```

A.4 Source code of the thread execution sequencing utility (`order.c`)

```c
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <pthread.h>
#include <semaphore.h>

char message[32];
sem_t ready;

void *writer(void *arg) {
    (void)arg;
    sleep(1);
    strcpy(message, "hello");
    printf("A: message written\n");
    sem_post(&ready);
    return NULL;
}

void *reader(void *arg) {
    (void)arg;
    printf("B: waiting for the message...\n");
    sem_wait(&ready);
    printf("B: I read \"%s\"\n", message);
    return NULL;
}

int main(void) {
    sem_init(&ready, 0, 0);
    pthread_t a, b;
    pthread_create(&b, NULL, reader, NULL);
    pthread_create(&a, NULL, writer, NULL);
    pthread_join(a, NULL);
    pthread_join(b, NULL);
    sem_destroy(&ready);
    return 0;
```

```
}
```

A.5 Source code of the bounded capacity resource utility (`parking.c`)

```
#include <stdio.h>
#include <unistd.h>
#include <pthread.h>
#include <semaphore.h>
#define PLACES 3
#define CARS   6

sem_t places;
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
int inside = 0;

void *car(void *arg) {
    long id = (long)arg;
    sem_wait(&places);
    pthread_mutex_lock(&lock);
    inside++;
    printf("car %ld enters (inside: %d)\n", id, inside);
    pthread_mutex_unlock(&lock);
    sleep(1);
    pthread_mutex_lock(&lock);
    inside--;
    printf("car %ld leaves (inside: %d)\n", id, inside);
    pthread_mutex_unlock(&lock);
    sem_post(&places);
    return NULL;
}

int main(void) {
    sem_init(&places, 0, PLACES);
    pthread_t t[CARS];
    for (long i = 0; i < CARS; i++)
        pthread_create(&t[i], NULL, car, (void *)(i + 1));
    for (int i = 0; i < CARS; i++)
        pthread_join(t[i], NULL);
    sem_destroy(&places);
    return 0;
}
```

A.6 Source code of the hardware atomic counter utility (`atomic.c`)

```
#include <stdio.h>
#include <pthread.h>
#include <stdatomic.h>
#define N 1000000

atomic_long counter = 0;

void *add(void *arg) {
    (void)arg;
    for (int i = 0; i < N; i++)
        atomic_fetch_add(&counter, 1);
    return NULL;
}

int main(void) {
    pthread_t a, b;
    pthread_create(&a, NULL, add, NULL);
    pthread_create(&b, NULL, add, NULL);
    pthread_join(a, NULL);
```

```
    pthread_join(b, NULL);
    printf("counter = %ld (expected %d)\n", (long)counter, 2 * N);
    return 0;
}
```

A.7 Build automation script (`Makefile`)

```
CC = gcc
CFLAGS = -Wall -pthread
SHELL = /bin/bash

PROGS = race mutex sem order parking atomic

all: $(PROGS)

race: race.c
    $(CC) $(CFLAGS) -o $@ $<

mutex: mutex.c
    $(CC) $(CFLAGS) -o $@ $<

sem: sem.c
    $(CC) $(CFLAGS) -o $@ $<

order: order.c
    $(CC) $(CFLAGS) -o $@ $<

parking: parking.c
    $(CC) $(CFLAGS) -o $@ $<

atomic: atomic.c
    $(CC) $(CFLAGS) -o $@ $<

test: all
    @./race && ./mutex && ./sem && ./order && ./parking && ./atomic

clean:
    rm -f $(PROGS)

.PHONY: all test clean
```